

PRAKTIKUM SISTEM OPERASI

MODUL 11

MEMORI XINU



Oleh:

Berlian Seva Astryana

2311104067

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

1. Buatlah perintah baru bernama freememory yang memiliki dua fungsi berikut:
 - a. Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

```
#include <xinu.h>

// Nama: Aulia Jasifa Br Ginting
// NIM: 2311104060

shellcmd xsh_freememory(int nargs, char *args[]) {

    struct memblk *blk;
    uint32 totalFreeSpace = 0;

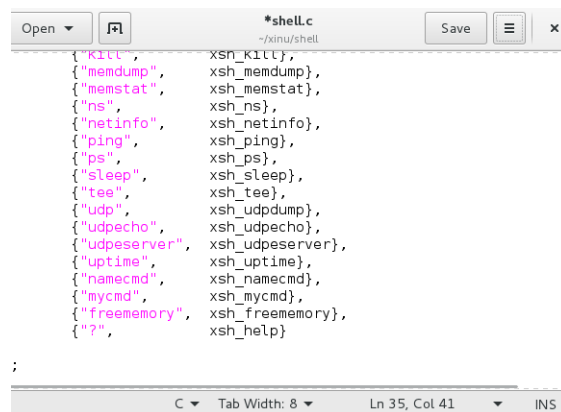
    kprintf("Free List:\n");
    kprintf("Block address Length (dec) Length (hex)\n");
    kprintf("-----\n");

    blk = memlist.mnext;

    while (blk != NULL) {
        kprintf("0x%08X %10d 0x%08X\n", (uint32) blk, blk->mLength, blk->mLength);
        totalFreeSpace += blk->mLength;
        blk = blk->mnext;
    }

    kprintf("\nTotal FreeSpace %d\n", totalFreeSpace);
    return 0;
}
```

Edit shell.c



Edit shprototypes.h

```

/* in file xsh_freememory.c */
extern shellcmd xsh_freememory| (int32, char *[]);

```

Output

```

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0010E110     -975128    0xFFFF11E8
0x00100000       5070848    0x004D6000
0x005E8000        57344     0x0000E000

Total FreeSpace 4153064
xsh $

```

2. Jawablah pertanyaan berikut:

- a. Mengapa Xinu memisahkan data segment dan BSS segment?

Xinu memisahkan data segment dan BSS segment agar penggunaan memori lebih efisien. Data segment menyimpan variabel global yang sudah memiliki nilai awal, sedangkan BSS menyimpan variabel global yang belum diinisialisasi. Dengan pemisahan ini, ukuran file program dapat menjadi lebih kecil.

- b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Saat memori dialokasikan, ukuran free space akan berkurang karena sebagian memori digunakan oleh proses. Sebaliknya, ketika memori didealokasikan, free space akan bertambah kembali. Perubahan ini berlangsung secara dinamis selama program berjalan.

- c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Heap lebih rentan menimbulkan masalah karena alokasi dan dealokasi dilakukan secara manual oleh program. Kesalahan pengelolaan dapat menyebabkan memory leak atau fragmentasi memori. Sementara itu, stack dikelola otomatis oleh sistem saat fungsi dipanggil dan selesai dieksekusi.

- d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Linked list memudahkan Xinu dalam menambah, menghapus, dan menggabungkan blok memori yang kosong. Struktur ini juga fleksibel karena ukuran blok memori dapat berbeda-beda. Selain itu, penggunaan linked list tidak memerlukan ukuran array yang tetap.

- e. Apa tantangan utama dalam penggunaan heap di Xinu!

Tantangan utama penggunaan heap di Xinu adalah fragmentasi memori akibat alokasi dan dealokasi yang berulang. Fragmentasi dapat menyebabkan ruang memori kosong tersebar sehingga sulit digunakan secara optimal. Selain itu, kesalahan pengelolaan heap dapat mengurangi efisiensi penggunaan memori.